

Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

Manik Malik 23169717

Assignment 6 – GPU

Q1.

To compute the peak double precision floating point performance of the node using different components, we need to consider the specifications provided. Let's calculate the peak performance for each case:

(a) CPUs: Each AMD Epyc CPU has 64 cores, and each core can execute 2 AVX2 FMAs per cycle at 2 GHz. Since AVX2 FMAs operate on 256-bit vectors, which is equivalent to 4 double precision numbers (64 bits each), the total peak performance per CPU can be calculated as follows:

Peak Performance per CPU = Number of cores × Super Scalarity × FMA factor × SIMD factor × Clock speed = 64 cores × 2 inst./cy × 2 AVX2 FMAs/core/cycle × 4Flops/Inst SIMD × 2 GHz = 2.048 TFLOPS

Since each node consists of 2 CPUs, the total peak performance for CPUs would be:

Total Peak Performance (CPUs) = Peak Performance per CPU × Number of CPUs = 2.048 GigaFLOPS × 2 = **4.096 TeraFLOPS**

(b) Single GPU: The A100 GPU has 108 Streaming Multiprocessors (SMs), and each SM can execute one double precision FMA per cycle at 1.41 GHz. Considering the warp size of 32, which means 32 double precision numbers can be processed in parallel, we can calculate the peak performance per GPU as follows:

Peak Performance per GPU = Number of SMs × Instructions per cycle × FMA factor × Clock speed × Warp size (SIMD) = 108 SMs × 1 instr/cycle × 1 FMA × 1.41 GHz × 32 = 108 FMA/cycle × 1.41 GHz × 32 = 4872.96 GigaFLOPS = **4.873 TeraFLOPS**

(c) All GPUs (without tensor units): Since each node consists of 8 A100 GPUs, the total peak performance for all GPUs would be:

Total Peak Performance (GPUs) = Peak Performance per GPU × Number of GPUs = 4.873 GigaFLOPS × 8 = 38983.68 GigaFLOPS = **38.98 TeraFLOPS**

Q2.

Basic idea

▪ Example: Matrix-matrix multiplication (DGEMM)

```
do i=1,N
  do j=1,N
    do k=1,N
      c(j,i)=c(j,i)+a(k,i)*b(k,j)
    enddo
  enddo
enddo
```

Minimum Balance Analysis

- Amount of work $\leftrightarrow$ 3 nested loops $\rightarrow 2 \times N^3 \times F$
- Minimum data transfer $\leftrightarrow$ 3+1 (NxN) matrices $\rightarrow (3 + 1) \times N^2 \times 8 B$

$$\rightarrow B_C = \frac{32 \times N^2}{2 \times N^3} \frac{B}{F} = \frac{16}{N} \frac{B}{F} \rightarrow O(N^{-1}) \quad (\text{Minimum balance})$$

PTFS 2023

May 15, 2023

34

The size of one matrix where $N = 1024$

Maximum data transfer $\Rightarrow \Rightarrow (3+1) \times 8 \text{ Bytes} \times 1024 \times 1024 = 33554432 \text{ Bytes} \sim 32 \text{ MB}$

Floating points needed in the complete Matrix – Matrix multiplication (DGEMM) = 2×1024^3
= $2 \times 10^9 = 2 \text{ Giga Flop}$

In Q1 we calculate the performance of CPU = 4.096 TerraFLOPS. = 4.096×10^{12}

Time taken by CPU = FLOPS for matrix multiplication / Pmax CPU = $2 \times 10^9 / 4.096 \times 10^{12} =$

Time taken by CPU = 0.5 milliseconds

In Q1 we calculate the performance of GPUs = 38.98 TerraFLOPS. = 38.98×10^{12}

Time taken by GPU = FLOPS for matrix multiplication / Pmax GPU = $2 \times 10^9 / 38.98 \times 10^{12}$

Time taken by GPU = 0.05 Milliseconds

If we calculate the speedup without data transfer time then the speed up comes to be around ~10
(0.5 milliseconds / = 0.05 Milliseconds)

But we need data transfer speed to be a factor so

Maximum bandwidth of read/write operations = 25 GB/s

Time takes for data transfer between CPU and GPU: $32\text{MB} / 25 \text{ GB/s} = 0.00128 \text{ Seconds}$.

This time will be added to the time taken by GPU = 0.05 milliseconds + 1.28 milliseconds

~ 1.33 milliseconds. Here there is no speedup infact **it makes using GPU ~ 3 times slower.**

Perhaps with higher values of N we may get to see a real benefit of the GPUs.

Q3:

Frontier

Peak FMA operations per cycle for CPU = Number of cores $\times$ Super Scalarity $\times$ FMA factor $\times$ SIMD factor $\times$ Clock Speed = 64 cores $\times$ 2 inst. /cy $\times$ 2 AVX2 FMAs $\times$ 4Flops/Inst SIMD = 1024 $\times$ 2 GHz (from part (a)) = **2.048 TFlops**

GPU:

Peak GPU performance = Number of SMs $\times$ Instructions per cycle $\times$ FMA factor $\times$ Clock speed $\times$ Warp size (SIMD) = 8 $\times$ 110 SMs $\times$ 1 instr/cycle $\times$ 1 FMA $\times$ 1.7 GHz $\times$ 64 = 95.744 TFlops

Performance per MI250 GPU = 11.968 TFlops

Peak GPU performance = **95.744 TFlops**

Q4:

The size of one matrix where $N = 1024$

Maximum data transfer $\Rightarrow \Rightarrow (3+1) \times 8 \text{ Bytes} \times 1024 \times 1024 = 33554432 \text{ Bytes} \sim 32 \text{ MB}$

Floating points needed in the complete Matrix – Matrix multiplication (DGEMM) = 2×1024^3
= $2 \times 10^9 = 2 \text{ Giga Flop}$

In Q3 we calculate the performance of CPU = 2.048 TerraFLOPS. = 2.048×10^{12}

Time taken by CPU = FLOPS for matrix multiplication / $P_{\max \text{ cpu}} = 2 \times 10^9 / 2.048 \times 10^{12} =$

Time taken by CPU = 1 millisecond

In Q3 we calculate the performance of GPUs = 95.744 TerraFLOPS. = 95.744×10^{12}

Time taken by GPU = FLOPS for matrix multiplication / $P_{\max \text{ gpu}} = 2 \times 10^9 / 95.744 \times 10^{12}$

Time taken by GPU = 0.0204 milliseconds

If we calculate the speedup without data transfer time then the speed up comes to be around ~ 50 times faster (1 milliseconds / = 0.02 Milliseconds)

But we need data transfer speed to be a factor so

Maximum bandwidth of read/write operations per GPU = 36 GB/s

Time takes for data transfer between CPU and GPU: 32MB/ 36 GB/s = 0.89 milliseconds.

This time will be added to the time taken by GPU = 0.02 milliseconds + 0.89 milliseconds

~ 0.91 milliseconds. The speedup here is **1.03 times** basically none.

As compared to Alex, Frontier is 1.5 times faster. (Total time taken by Alex GPU/ Total time taken by Frontier GPU = $1.33/0.91$)

Q5:

To determine the minimum number of threads required to saturate the DRAM interface, we need to consider the DRAM interface bandwidth and the memory access pattern of the kernel.

The DRAM interface bandwidth is given as 1555 GB/s

To saturate the DRAM interface, we need to ensure that the memory transactions of all threads collectively utilize the full bandwidth.

Memory access time = $670 \text{ cycles} / 1.41 \text{ GHz} = 475 \text{ ns}$

Parallelism = $1555 \text{ GB/s} * 475 = 738625 \text{ bytes can be transferred in } 475 \text{ ns}$

Assuming double precision elements (8 bytes per element), each thread requires 4 memory transactions: In our case the function there are 3 loads and 1 store so 32 bytes are transferred per 475 ns latency

Let N be the total number of threads. The total memory transactions per second can be calculated as Each thread issues load operations independently

$32N = 738625$

No. of threads = 23082 threads are required to saturate the DRAM interface

(b)

To minimize cache traffic, it is recommended to choose a thread block size that aligns with the GPU's memory hierarchy. This allows for efficient memory accesses and minimizes cache conflicts. The goal is to improve data locality and minimize cache misses.

In the given kernel code, the thread block size is determined by the dimensions of the grid (blockDim.x and blockDim.y). To minimize cache traffic, you can choose a thread block size that is a multiple of the GPU's warp size, typically 32 threads.

By using a thread block size that aligns with the warp size, you can ensure that memory accesses within a warp are coalesced and minimize cache traffic.

- Choose a thread block size that is a multiple of the warp size. For modern GPUs, the warp size is typically 32 threads.
- Consider the shared memory capacity of the GPU. Utilizing shared memory effectively can help reduce global memory accesses. Choosing a thread block size that allows for efficient use of shared memory can improve performance.
- Consider the number of registers used by each thread. Choosing a smaller thread block size can reduce the register usage per thread and increase the occupancy.

Q6: Using the skeleton code provided in CUDA-Stream directory, we completed the given GPU kernel as shown below

```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Kernel
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

__global__ void gpuKernel( double *d_C, double *d_A, double *d_B, int DATA_N ) {

    for(int pos=(blockIdx.x*blockDim.x)+threadIdx.x;
        pos < DATA_N;
        pos += blockDim.x*gridDim.x) {

        d_C[pos] = d_A[pos] + d_B[pos];
    }
}

```

We then accessed Alex and executed the given command as show below;

```
ptfs231h@alex2:~/ptfs_23/CUDA_Mod$ stree -N 1 --gres=gpu:a100:1 -p a100 -C a100_80 --time 4:00:00 --pty /bin/bash -l
## Starting TaskPrologue of job 761379 on a0537 at Thu Jun 15 10:38:35 CEST 2023
Running on cores 96-111 with governor ondemand
Thu Jun 15 10:38:35 2023

-----+
| NVIDIA-SMI 530.30.02                Driver Version: 530.30.02    CUDA Version: 12.1      |
|-----+-----+-----+-----+-----+-----+-----+-----+
| GPU  Name                               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf                Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |                      |
|-----+-----+-----+-----+-----+-----+-----+
|   0   NVIDIA A100-SXM4-80GB              On | 00000000:CC:00:0 Off |                    0 |
| N/A   31C    P0               60W / 400W|  0MiB / 81920MiB |      0%    Default |
|                                           |                    | Disabled |
|-----+-----+-----+-----+-----+-----+
|
| Processes:
| GPU   GI    CI          PID    Type    Process name                        GPU Memory
|      ID    ID
|-----+-----+-----+-----+-----+-----+
| No running processes found
|-----+-----+-----+-----+-----+
## Finished TaskPrologue
ptfs231h@a0537:~/ptfs_23/CUDA_Mod$

ptfs231h@a0537:~/ptfs_23/CUDA_Mod$ ls
bench.sh  Makefile  Stream  Stream.cu
ptfs231h@a0537:~/ptfs_23/CUDA_Mod$ ./Stream 10000 16 32
Initializing data...
...allocating CPU memory.
...allocating GPU memory.
...generating input data in CPU mem.
...copying input data to GPU mem.
Data init done. Now executing GPU kernel ...
Reading back GPU result...
CUDA performance: RAW: 195.995514 MIt/s      incl. transf.: 68.422577 MIt/s
Shutting down...
ptfs231h@a0537:~/ptfs_23/CUDA_Mod$
```

As suggested, I experimented with different values for the thread starting from 64.

```
./Stream 1000000 108 64
```

```
CUDA performance: RAW: 9425.402247 MIt/s    incl. transf.: 815.536457 MIt/s
```

```
./Stream 1000000 108 128
```

```
CUDA performance: RAW: 14513.162630 MIt/s    incl. transf.: 839.028606 MIt/s
```

```
./Stream 1000000 108 192
```

```
CUDA performance: RAW: 11781.752809 MIt/s    incl. transf.: 817.762527 MIt/s
```

```
./Stream 1000000 108 256
```

```
CUDA performance: RAW: 15141.891697 MIt/s    incl. transf.: 840.373472 MIt/s
```

```
./Stream 1000000 108 320
```

```
CUDA performance: RAW: 13148.288401 MIt/s    incl. transf.: 834.853503 MIt/s
```

```
./Stream 1000000 108 384
```

```
CUDA performance: RAW: 16644.063492 MIt/s    incl. transf.: 648.470006 MIt/s
```

```
./Stream 1000000 108 448
```

```
CUDA performance: RAW: 16644.063492 MIt/s    incl. transf.: 834.023464 MIt/s
```

```
./Stream 1000000 108 512
```

```
CUDA performance: RAW: 17848.102128 MIt/s    incl. transf.: 856.155134 MIt/s
```

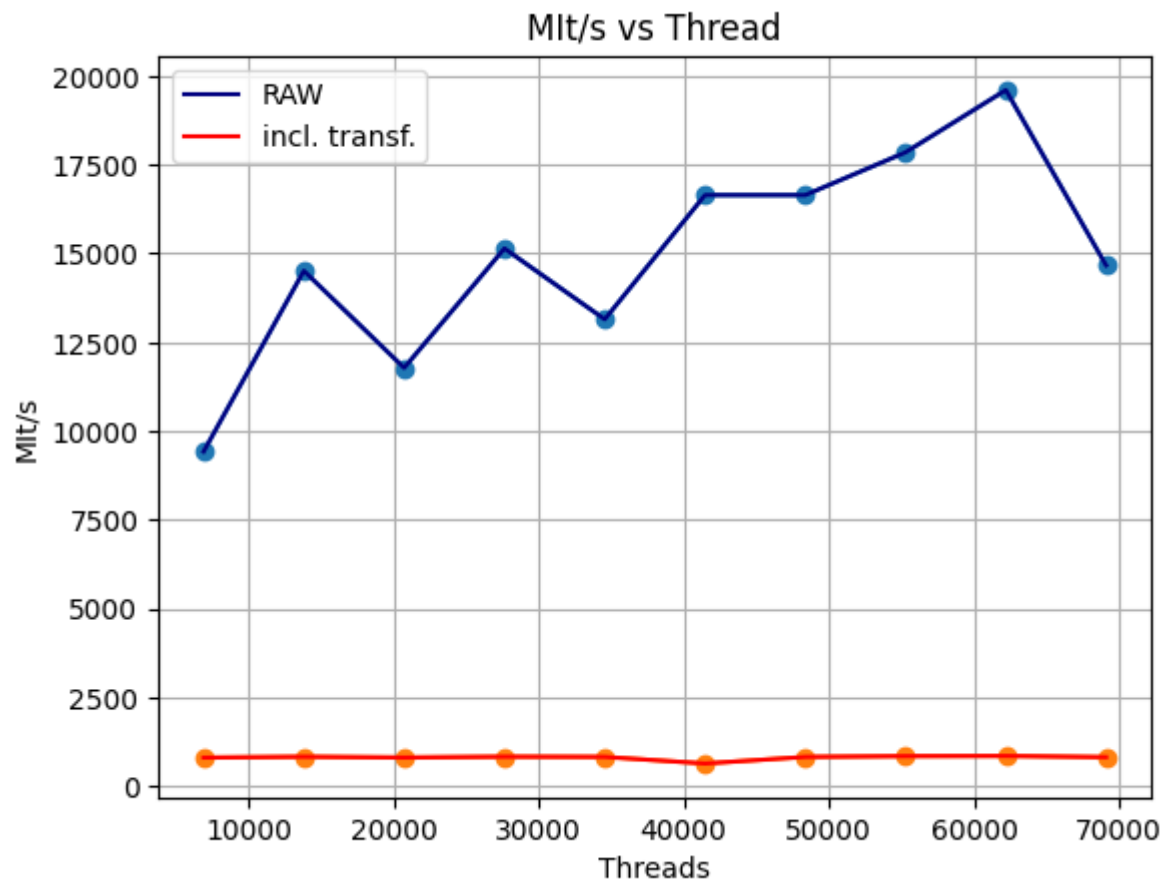
```
./Stream 1000000 108 576
```

```
CUDA performance: RAW: 19599.551402 MIt/s    incl. transf.: 861.253388 MIt/s
```

```
./Stream 1000000 108 640
```

```
CUDA performance: RAW: 14665.398601 MIt/s    incl. transf.: 827.768699 MIt/s
```

We plotted the obtained data and our graph is shown below



We also observed that the max value of the array length to get maximum bandwidth is within 100000000-1000000000

(b) We estimate bandwidth of the PCI Express to be within 800 MIt/s